

KKluaverb Package Documentation

Kosei Kawaguchi a.k.a. KKTeX

Version 2.3.0 (2026/06/12)

目次

1	Outline	3
2	Acknowledgements / Credit	3
3	Dependencies	3
4	Basic Usage of Lua-enhanced <code>\verb</code>	3
4.1	Fundamental Behavior	3
4.2	In Verbatim Environments	4
4.3	Optional Usage	4
4.4	Behavior in TOC, Index etc.	5
5	Basic Usage of Lua-enhanced <code>\lstlisting</code>	5
5.1	Fundamental Behavior	5
5.1.1	<code>\KKvLNChange</code>	6
5.2	Additional Description	7
5.2.1	Note#1	7
5.2.2	Note#2	8
5.2.3	Note#3	8
5.2.4	Note#4: Automatic Line Wrapping (Since v2.3.0)	8
6	Color Mapping	9
6.1	Basic Behavior	9
6.1.1	<code>word_boundary</code> , <code>word_components</code> options	10
6.1.2	<code>comment_char</code> , <code>comment_color</code> , <code>escape_char</code> options	10
6.1.3	<code>delimiters</code> , <code>forced_token</code> options	11
6.2	Example	11
7	Text Mapping	13
7.1	TeX-command Mapping (Since v2.3.0)	14

1 Outline

The `KKluaverb` is a LaTeX package which provides a Lua-enhanced `\verb` command, `\KKverb`. It can be used in the table of contents, index, underline commands (e.g., `\underlineKK` provided by the `luwa-ul` package), `tblr` environment and so on.

2 Acknowledgements / Credit

In developing this package, I made use of an algorithm which is used in “`bxcrawstr`” (by Takayuki YATO)¹⁾.

3 Dependencies

This package internally uses the following packages:

- `luatexbase`, `luacode`
- `pgfkeys`
- `xcolor`

4 Basic Usage of Lua-enhanced `\verb`

4.1 Fundamental Behavior

As mentioned in the “Outline” section, this package mainly provides enhanced `\verb` command. The usage is not much different from the normal one.

Input
<pre> \KKverb \def\TEST{Test Text.} </pre>
Output
<pre>\def\TEST{Test Text.}</pre>

In the argument of the command, any linebreaks and blank lines are ignored. However, every space are preserved and rendered exactly as it appears.

1) package: <https://gist.github.com/zr-tex8r/c7901658a866adfdc3cd66b6dfa86997>
article: <https://zrbabbler.hatenablog.com/entry/20181222/1545495849>

4.2 In Verbatim Environments

When this package is loaded, the Lua-scanning which enables `\KKverb` to detokenize its argument is activated. However, when `\KKverb` is used in verbatim environments such as “`lstlistings`” and “`tcblistings`”, the argument cannot be decoded properly. In such cases, you should use `\KKvScanOff` which deactivates the scanning process used internally in `KKverb.lua`. Of course, `\KKvScanOn` re-activates the process.

4.3 Optional Usage

In addition to the main function of `\KKverb`, some optional functions are provided.

`\KKvOpChange` can change font and color of the output of `\KKverb`. Also, it can activate or deactivate the scanner. The usage is as follows:

Input	Output
<pre> 1 {\KKvOpChange{color=blue, font=\gtfamily, enabled=true}% 2 \KKverb \def\TEST{Test Text.} } </pre>	<pre> \def\TEST{Test Text.} </pre>

表 1: `\KKvOpChange`

Key	Default Value	Description
<code>font</code>	<code>\ttfamily</code>	Sets the font for the verbatim text.
<code>color</code>	<code>black</code>	Sets the text color.
<code>enabled</code>	<code>true</code>	Activates or deactivates the scanning engine.
<code>wrap</code>	<code>false</code>	Enables auto wrapping in code blocks. (Since v2.3.0)
<code>wrapindent</code>	<code>2em</code>	Continuation line indent. (Since v2.3.0)

Additionally, the `enabled` option functions the same as `\KKvScanOn` and `\KKvScanOff`.

Also, you can change delimiters by using `\KKvSetDelims` as follows in order to change the delimiters of the command:

Input
<pre> 1 \KKvSetDelims{[<]{>}} 2 </pre>

```
3 Changed the delimiters: \KKverb[<\def\foo\s>]
```

Output

```
Changed the delimiters: \def\foo\s
```

By default, the delimiters are set as ”|”. So you can reset them by this:

Input

```
| \KKvSetDelims{|}{|}
```

4.4 Behavior in TOC, Index etc.

One of the strong points of `\KKverb` is that it can be used in table of contents, index, and footnotes and more. While the traditional `\verb` command always causes an error in such contexts, `\KKverb` does not. This is achieved through the following logic, using the TOC as an example.

When the TeX system generates the TOC, it produces an auxiliary file with the extension `.toc`. In this file, entry data is typically stored in an expanded state.

However, this package requires the raw text data to be passed unexpanded. This is because the text must be intercepted and processed by `process_input_buffer` before TeX’s tokenizer converts it into tokens. To resolve this, I implemented a mechanism that automatically prepends the `\unexpanded` command to the starter flag when writing to auxiliary files:

```
1 % In TOC
2 \noexpand\KKlvStart*<encoded tests>\noexpand\KKlvEnd*%
```

Then, the content of the `.toc` file is processed by TeX in the usual way subsequently, because the `\unexpanded` is no longer present.

5 Basic Usage of Lua-enhanced `lstlisting`

5.1 Fundamental Behavior

`\KKcodeS` and `\KKcodeE` provide an environment-like output which is very close to the `lstlisting`. The usage is very simple. The code between `\KKcodeS` and `\KKcodeE` is rendered as raw texts.

If you want to attach linenumbers to the left of the each line, `\KKcodeS+` will meet the request. Please note that you do not need to type `\KKcodeE+`. `\KKcodeS+` and `\KKcodeE` is the proper pair.

Input

```
1 \KKcodeS+
2 % A sample command
3 \long\def\myprog#1{%
4   \ifx#1\empty
5     \relax
6   \else
7     \message{Processing...}%
8     #1
9   \fi
10 }
11 \KKcodeE
```

Output

```
1 % A sample command
2 \long\def\myprog#1{%
3   \ifx#1\empty
4     \relax
5   \else
6     \message{Processing...}%
7     #1
8   \fi
9 }
```

When you use this Lua-based environment, please keep the following points in mind:

- All indents and spaces are displayed.
- Line breaks and blank lines are rendered exactly as they appear in the source.
- The delimiter characters of `\KKverb` have no special meaning here (since v2.2.0): the environment is terminated only by `\KKcodeE`.

5.1.1 \KKvLNChange

This command can change the style of the linenumbers.

表 2: \KKvLNChange

Key	Default Value	Description
font	\ttfamily	Sets the font for the linenumbers.
color	black!80	Sets the color of the linenumbers.
size	\small	Set the size of the linenumbers.
start	1	Set the start number.
style	0	Set the style.

Please note that you do not have to use style key when you use this package in a normal way. This is because style1 and style2 are completely covered by \KKcodeS/E or \KKcodeS+/E environment. The following example illustrates this point.

Input

```

1 {\KKvLNChange{style=1}
2 \KKverb|
3 % contents
4 |}
5
6 \KKcodeS
7 % contents
8 \KKcodeE

```

The two above produce identical output. To summarize,

style=0 This is the normal usage of the \KKverb.

style=1 This style corresponds to \KKcodeS/E environment.

style=2 This style corresponds to \KKcodeS+/E environment.

5.2 Additional Description

Some corner cases should be payed attention.

5.2.1 Note#1

Since v2.2.0, the raw scan remembers which command started it. A scan started by \KKverb is terminated only by its closing delimiter, and a scan started by \KKcodeS is terminated only by \KKcodeE. Therefore you can now write \KKcodeS and \KKcodeE in the argument of \KKverb, and the delimiter characters of \KKverb can appear freely inside the \KKcodeS/E environment. (In versions prior to v2.2.0, \KKcodeS forcibly started the encoding system even when it was wrapped by \KKverb, and the \KKverb delimiter terminated the \KKcodeS/E environment.) Note that \KKcodeS

appearing outside of a raw scan still starts the encoding system unconditionally; if you need to display it as it is in a normal context, use `\KKverb` or deactivate the scanner by `\KKvScanOff`.

5.2.2 Note#2

Any text on the same line that falls "inside" the `\KKcodeS` and `\KKcodeE` boundaries is ignored, while text "outside" them is preserved. The following example will illustrate the behavior.

Input
1 Not ignored. \KKcodeS+ Ignored. 2 % Contents 3 Ignored. \KKcodeE Not ignored.

Output
Not ignored. 1 % Contents Not ignored.

5.2.3 Note#3

Since v2.2.0, you can output a literal `\KKcodeE` inside the environment by putting the escape character (default: a backslash) immediately before it. The escape character itself is removed from the output, and the raw scan continues.

Input
1 \KKcodeS 2 literal: \KKcodeE is rendered as raw text 3 \KKcodeE

Output
literal: \KKcodeE is rendered as raw text

The escape character can be changed by `\KKvSetCodeEscape{<character>}`.

5.2.4 Note#4: Automatic Line Wrapping (Since v2.3.0)

By default, long lines in `\KKcodeS/E` blocks overflow beyond the right margin (Overfull `\hbox`). Since v2.3.0, you can enable automatic wrapping by `\KKvOpChange{wrap=true}`.

Input

```

1 \KKvOpChange{wrap=true}
2
3 \KKcodeS
4 x = aaaa + bbbb + cccc + dddd + eeee + ffff + gggg + hhhh + iiii + jjjj +
   kkkk
5 \KKcodeE
6
7 \KKvOpChange{wrap=false}

```

Output

```

x = aaaa + bbbb + cccc + dddd + eeee + ffff + gggg + hhhh + iiii + jjjj + kkkk

```

The indentation of continuation lines equals the original code indentation plus `wrapindent` (default: 2em). For style=2 (line-numbered), continuation lines do not receive a line number.

6 Color Mapping

6.1 Basic Behavior

This package provides color-mapping option. You can specify the color of the keyword-color, comment-color, comment-token, etc. In order to set a colormap, you should use `luacode*` environment as follows:

Input

```

1 \begin{luacode*}
2   KKLuaVerb.preset["<name-of-style>"] = {
3     map = {
4       <color1> = { "<token1>", "<token2>",... },
5       <color2> = { "<token1>", "<token2>",... },
6       ...
7     },
8
9     options = {
10      word_boundary = true or false,
11      word_components = "[<word-components>]",
12      comment_char = "<comment-token>",

```

```

13     comment_color = "<comment-color>",
14     escape_char = "<escape-token>",
15     delimiters = {
16         { start = '<starter1>', stop = '<ender1>', color = "<patial-color1>" },
17         { start = '<starter2>', stop = '<ender2>', color = "<patial-color2>" },
18     },
19     forced_tokens = {
20         ["<forced-token1>"] = "<color2>",
21         ["<forced-token2>"] = "<color2>",
22     }
23 }
24 }
25 \end{luacode*}

```

The “color” parameter here refers to those defined in the `xcolor` package.

You can set multiple presets, and can change or activate by `\KKvUsePreset{<name-of-style>}` command.

6.1.1 `word_boundary`, `word_components` options

These two options are closely related to each other. In the argument of the `\KKverb` command or inside of the `\KKcodeS/E` environment, the mapped keywords are colored as you specified. However, if the string is included in the other words, you need to prevent the system provided from wrongly color the string. For example, in the situation that `\def` command is mapped as a keyword but `\define` is not, the part of `\def` in `\define` should not be colored.

For this purpose, I made a “word_boundary detector”. In advance, set the components of the “word” like this:

```
1 word_components = "[A-Za-z0-9_]",
```

Then, activate the `word_boundary` detector:

```
1 word_boundary = true,
```

By all this, any characters or tokens except for “[A-Za-z0-9_]” works as a separator of the words or tokens, and each keywords are properly colored.

By default, `word_boundary` are set as “true”, and `word_components` are set as “[A-Za-z0-9_]”.

6.1.2 `comment_char`, `comment_color`, `escape_char` options

These options are provided in order to set comment character, comment color, and escape character respectively.

If you set `comment_char`, the following texts in the same line are colored as `comment_color`. But, if you put the `escape_char` before the `comment_char`, the very token and the following texts are not

colored as `comment_color`.

6.1.3 delimiters, forced_token options

These options are provided for the purpose of setting `delimiter_token` and the color of the texts between the tokens. Additionally, you can set multiple `forced_token`. These are forced to be colored as you specify, even they are between the delimiters.

6.2 Example

Input Setting

```
1 \begin{luacode*}
2   KKLuaVerb.preset["TeXStyle"] = {
3     map = {
4       DarkCyan = { "{", "}" },
5       OrangeRed = { "[", "]" },
6       DeepPink = { "(", ")" },
7       DarkGoldenrod = { "&", "$", "^", "_" },
8
9       CornflowerBlue = {
10        "\\documentclass", "\\usepackage", "\\begin", "\\end",
11        "\\section", "\\subsection", "\\chapter",
12        "\\makeatletter", "\\makeatother", "\\ExplSyntaxOn", "\\ExplSyntaxOff"
13      },
14
15      MediumPurple = {
16        "\\def", "\\edef", "\\gdef", "\\xdef",
17        "\\newcommand", "\\renewcommand", "\\providecommand",
18        "\\let", "\\setcopy", "\\long", "\\global"
19      },
20
21      Magenta = {
22        "\\if", "\\else", "\\fi", "\\ifx", "\\ifdefined",
23        "\\ifnum", "\\ifdim", "\\loop", "\\repeat", "\\noexpand"
24      },
25
26      IndianRed = {
27        "\\KKlvStart*", "\\KKlvEnd*"
28      }
29    },
30  }
```

```

30
31 options = {
32   word_boundary = true,
33   comment_char = "%",
34   comment_color = "ForestGreen",
35   escape_char = "\\ ",
36   delimiters = {
37     { start = "$", stop = "$", color = "SpringGreen3" },
38     { start = "\\[", stop = "\\]", color = "SpringGreen3" },
39   },
40   forced_tokens = {
41     ["{"] = "DarkCyan",
42     ["}"] = "DarkCyan",
43   },
44 }
45 }
46 \end{luacode*}

```

Input

[Actual Use](#)

```

1 \KKvUsePreset{TeXStyle}
2
3 \KKcodeS+
4 \usepackage{KKluavverb}
5
6 \begin{document}
7   Hello, KKTex!
8 \end{document}
9 \KKcodeE
10
11 \KKcodeS
12 { [ \ ( \% \$ # & _ ^ ) / ] }
13 Inline mathmode: $a + b = \frac{x}{y}$
14 Display-style mathmode:
15 \[a + b = \frac{x}{y}\]
16 \KKcodeE

```

Output

```

1 \usepackage{KKluavverb}
2
3 \begin{document}
4   Hello, KKT $\text{E}$ X!
5 \end{document}
{ [ \ ( \% \$ # & _ ^ ) / ] }
Inline mathmode:  $a + b = \frac{x}{y}$ 
Display-style mathmode:

$$[a + b = \frac{x}{y}]$$


```

7 Text Mapping

By using `\KKvSetMap`, you can replace a certain text in the argument of `\KKverb` and `\KKcodeS/E` environment.

Input

Actual Use

```

1 \KKvSetMap <{!}<{ EXCLAMATION}
2 \KKvSetMap <{?}<{ QUESTION}
3
4 \KKverb|Wait, you really change them!?!|
5
6 \KKvSetMap{!}{!}
7 \KKvSetMap{?}{?}
8 \KKverb|Oh, how can I reset them!?!|

```

Output

```

Wait, you really change them <EXCLAMATION> <QUESTION>
Oh, how can I reset them!?!

```

Using this, you can visualize all spaces in the argument of `\KKverb` and `\KKcodeS/E` environment. However, you have to use `\KKvSpaceForce` if you want to reset. By default, this package automatically replaces any spaces to Unicode character U+00A0 (Non-breaking Space). Therefore, you need to use the special command if you want to quit visualizing spaces.

Input	Actual Use
<pre> 1 \KKvSetMap{ }{<sp>} 2 \KKverb Look! All spaces are visualized!! 3 4 \KKvSpaceForce 5 \KKverb Oh, spaces are no longer visualized. </pre>	
	<pre> <sp><sp>Look!<sp>All<sp>spaces<sp>are<sp>visualized!!<sp><sp> Oh, spaces are no longer visualized. </pre>

7.1 TeX-command Mapping (Since v2.3.0)

`\KKvSetMap` replaces a character with another character. However, there are cases where you want to replace a character with a TeX command. For example, `\KKvSetMap{ }{\_}` replaces spaces with U+2423, but this character may be rendered as a full-width glyph in Japanese font contexts.

`\KKvSetMapTeX{<from>}{<tex-cmd>}` solves this by mapping the character to a TeX command, which is emitted via `tex.sprint` (with expansion) at output time.

Input	Actual Use
<pre> 1 \KKvSpaceVisible % ON: spaces -> \textvisiblespace 2 \KKlvStart**20*20Hello*20World*21*20*20\KKlvEnd* 3 4 \KKvClearMapTeX{ } % OFF: back to NBSP 5 \KKlvStart**20*20Hello*20World*21*20*20\KKlvEnd* </pre>	
	<pre> Hello World! Hello World! </pre>

`\KKvSpaceVisible` is a convenience wrapper for `\KKvSetMapTeX{ }{\textvisiblespace}`. `\KKvClearMapTeX{<char>}` removes the TeX-command mapping for a character and restores the default behaviour (for space: reverts to non-breaking space). Note that calling `\KKvSetMap` on a character that currently has a TeX-command mapping automatically removes the old mapping.